

TRADEVISION AI

GOD DOC

The complete blueprint to build, deploy, and WIN Builderr.ai

Every rule · Every strategy · Every line of code · Every formula · Every trap

Contest	Builderr.ai Trading Agent Challenge — Round 1
Deadline	July 2, 2026 — 30 days live forward test
Prize Pool	\$2,000 USD \$1,200 / \$500 / \$300 (top 3) + bonus from backers
Bonus	#1 bot trades a REAL \$100,000 Nasdaq account. Public P&L.;
Entry Fee	FREE. No brokerage. No API key. No money needed.
Revisions	Up to 4 re-submissions during Round 1. Submit early.
Scoring	Calmar Ratio = Annualized Return / Max Drawdown
Stack	React 18 + FastAPI (Python) + yfinance + Recharts
Developer	Meet — Solo — Gandhinagar, India
Doc Version	1.0 — Compiled June 2026 from official sources

This document contains every detail you need to submit a winning bot and build a world-class trading dashboard — sourced directly from Builderr.ai official guidelines, build guide, and template repository. Nothing is assumed. Everything is defined.

TABLE OF CONTENTS

0 1	OFFICIAL CONTEST RULES	Prize, timeline, DQ list, what's allowed — verbatim from builderr.ai
0 2	SCORING SYSTEM DEEP DIVE	Calmar ratio, admission, forward test, re-run, how to optimize
0 3	THE HARD CONSTRAINTS	Every leverage cap, position rule, trade limit — with exact numbers
0 4	WINNING STRATEGY ANALYSIS	All 5 official starter ideas + 3 deep strategies with code
0 5	FULL agent.py CODE	Production-ready decide() — dual momentum + vol targeting + risk-off
0 6	BACKTESTING ENGINE	Full backtest.py, metrics.py, data.py — copy-paste ready
0 7	FASTAPI BACKEND	Complete main.py, all endpoints, request/response shapes
0 8	REACT FRONTEND — ALL SCREENS	Dashboard, Backtest, Strategy, Signals — full component code
0 9	DATABASE DESIGN	SQLite schema for storing backtest results, trade history
1 0	API CONTRACTS	Every endpoint, payload, response — typed and documented
1 1	DEPLOYMENT GUIDE	Railway + Vercel — step by step, free tier, env vars
1 2	SUBMISSION CHECKLIST	Every step from clone to email — nothing missing
1 3	TRAPS & DISQUALIFICATIONS	Every way to fail — with prevention for each
1 4	FINANCE KNOWLEDGE FOR THIS CONTEST	Only the concepts that matter for winning Calmar score
1 5	7-DAY BUILD PLAN	Hour by hour — what to build when

01 — OFFICIAL CONTEST RULES

Sourced directly from builderr.ai/guidelines — June 2026

What This Is

A trading-bot competition. You write a bot, Builderr runs it on real market data, the best one wins. \$2,000 prize pool split 60/25/15 among top 3 (\$1,200 / \$500 / \$300). If backers grow the pool, the extra splits in the same ratio. The winner's code also runs on a real \$100k Nasdaq account after the contest, with weekly P&L; posted publicly via a live ticker.

Prize Structure

Place	Prize (USD)	Prize (INR approx)	What else you get
1st	\$1,200	~Rs. 1,00,000	Code trades REAL \$100k Nasdaq. Public P&L; forever.
2nd	\$500	~Rs. 41,500	Shared with backers only if they paid \$200+
3rd	\$300	~Rs. 25,000	Shared with backers only if they paid \$200+
4th+	\$0	—	Nothing shared. You keep your code 100%.

Timeline — Exact Dates

Round 1 Opens	Tuesday June 2, 2026 — 7:30 AM IST (Monday June 1, 10:00 PM ET)
Round 1 Closes	Wednesday July 2, 2026 — 30 days of live forward testing
Submit NOW	Submissions open right now. Submit early to maximize live trading window.
Max Revisions	4 re-submissions during Round 1. Each revision re-runs admission.
Results	Re-run on top finishers → winners announced after July 2
Next Round	If nothing wins, prize rolls to next round

Who Can Enter

Eligibility	ANYONE, ANYWHERE in the world. Including India. Including students.
Team Size	Solo OR a pair. Your choice.
One Exception	If you know the exact hidden test dates, you can still play but exhibition-only (no prize).
Cost	Completely FREE. No brokerage login. No deposits. No API keys required.

What Gets You DISQUALIFIED — Complete List

■ These are the ONLY ways to be banned. Know them all cold.

Lookahead bias	Using data your bot couldn't have had at decision time. Example: using today's close to decide today. Caught by code review + re-run. INSTANT DQ.
Breaking hard caps	Gross leverage >1.5x OR single position >30% for more than 5 days. Engine auto-flattens AND you're out.
>50% blowup	If bot loses more than 50% in the admission safety check, you won't be admitted. Not a DQ — just not admitted.
Malicious code	Any code designed to harm, abuse, or game the system.
Multiple identities	Gaming with multiple accounts. Clear bad faith.
Bad faith (catch-all)	Any obvious attempt to exploit the system not listed above. Used narrowly, never to penalize honest bots.

■ You are NEVER DQ'd for: losing, finishing low, having a simple bot, using external data/news, revising strategy, or being a beginner.

What Does NOT Get You Disqualified

- Losing, or finishing last place — perfectly welcome
- A 'fair-weather' bot that fails in crashes — admission is not a skill gate
- Using legitimate external data — news feeds, prices, your own ML models
- A dead-simple bot — baseline.py (5 lines) is a valid, admissible submission
- Revising up to 4 times during Round 1
- Changing your whole strategy between cohorts
- Using an LLM (Claude, GPT) to generate trade signals — just don't commit the API key

Code Ownership — Who Owns What

Your code	You keep 100% ownership. Template is MIT licensed. Builderr never resells or claims your strategy.
If top 3	Your winning agent is SHARED with people who backed the bounty. #1 trades the real \$100k. You agreed to this by entering.
If not top 3	NOTHING of yours is shared with anyone. Total privacy.
Key safety	NEVER commit API keys to a public repo — if public, the whole internet sees it. Use private repo or endpoint mode.

02 — SCORING SYSTEM DEEP DIVE

The 3-Step Judging Process

Step 1: Admission	Safety check ONLY. Bot runs against 3 hidden past periods (a crash, slow decline, vol spike). Must run cleanly, respect caps, no >50% blowup. DOES NOT test skill — only weeds out broken/reckless bots.
Step 2: Live Test	THE ACTUAL RANKING. Every admitted bot trades live for 30 days (June 2 – July 2) on same paper account, same data, same fills. Ranked by Calmar Ratio. Future data nobody has seen.
Step 3: Re-run	LUCK CHECK. Top finishers re-run on other market windows they never saw. Skill repeats; luck doesn't. This is how winners are confirmed.

Calmar Ratio — The ONLY Score That Matters

$$\text{Calmar Ratio} = \text{Annualized Return} / \text{Max Drawdown}$$

$$\text{Annualized Return} = (\text{End_Value} / \text{Start_Value})^{(252/\text{trading_days})} - 1$$

$$\text{Max Drawdown} = \max[(\text{Peak_Value} - \text{Trough_Value}) / \text{Peak_Value}]$$

■ This is not about making the most money. It's about making money WITHOUT a deep hole.

Calmar Examples — What Beats What

Return	Max Drawdown	Calmar	Verdict
+10%	-2%	5.00	EXCELLENT — wins even against much higher returns
+20%	-5%	4.00	Very strong
+15%	-6%	2.50	Good — competitive
+30%	-15%	2.00	Decent but risky
+50%	-30%	1.67	Mediocre — big drawdown hurts badly
+30%	-25%	1.20	Weak — loses to +10%/-2% bot easily
+100%	-60%	1.67	Looks impressive, actually mediocre
+8%	-1.5%	5.33	Simple SPY-above-SMA bot could do this

KEY INSIGHT: A boring bot that stays in cash during downturns and earns 8-12% with minimal drawdown beats a flashy bot making 50% with big swings. Design for CALMAR, not raw return.

What Admission Actually Tests

3 Hidden Periods	A market crash (like March 2020 type), a slow decline (like late 2022), a vol spike (like August 2024).
------------------	---

Pass Criteria	Bot must: run without errors, respect leverage cap, not blow up >50%.
Robustness Profile	You get an email back (same day, usually hours) showing how your bot behaved on each hidden period.
Timing Matters	Earlier submission = more days in the live 30-day window. Submit now even with baseline.py.
Local Preview	Run: <code>python preview.py</code> — shows PASS/FAIL on real sample windows in ~10 seconds. Do this before every submission.

03 — HARD CONSTRAINTS (EXACT NUMBERS)

■ These are auto-enforced. Break them and the engine flattens your position AND you may be DQ'd.

Rule	Hard Limit	What happens if broken	How to stay safe
Long only	No short-selling (v0)	Order rejected silently	Only side='buy' or side='sell' existing positions
Gross leverage	$\leq 1.5 \times$ your equity (beta-adj)	Engine auto-flattens you	Never exceed 1.5x total exposure
Concentration	No single position $>30\%$ for >5 consecutive trading days	Auto-flatten + DQ risk	Keep each position $\leq 28\%$ to be safe
Trades per day	≤ 50 orders	Excess orders ignored	Typical bots make 0-5 trades/day
Minimum hold	≥ 60 seconds	Order rejected	Daily bars = always fine
decide() runtime	≤ 5 seconds per call	Call times out, no orders	Test locally: time your decide()
Off-universe tickers	Silently ignored	Order lost, cash unused	Only trade tickers in universe.json

Leverage Beta Multipliers — Critical for Calculation

3x multiplier TQQQ, SOXL, UPRO, SPXL, TNA, FAS, TECL, LABU, CURE, DRN, UDOW, NAIL

2x multiplier QLD, SSO, DDM, ROM, UWM, AGQ

1x multiplier Everything else — SPY, QQQ, individual stocks, sector ETFs

Gross Exposure = $\Sigma(\text{position_value} \times \text{beta_multiplier})$

SAFE: 50% TQQQ (3x) + 50% cash = 1.5x gross = OK (right at limit)

UNSAFE: 60% TQQQ (3x) = 1.8x gross = AUTO-FLATTEN

SAFE: 100% SPY (1x) + 100% QQQ (1x) = impossible ($>100\%$), but each at 50% = 1.0x

■ Never hold more than 25% TQQQ or 33% QLD. These eat your leverage budget fast. Stick to 1x ETFs for safety.

The Universe — What You Can Trade

Top ~1000 US names by liquidity — frozen at round open (same list for everyone). See universe.json in the template repo for full list. Key names to know:

Category	Tickers	Notes
Mega Cap Tech	AAPL MSFT NVDA AMZN META GOOGL TSLA	Highest liquidity, 1x
Semis	AMD AVGO MU MRVL QCOM	High momentum, 1x
AI/Chips	PLTR COIN	High vol, momentum plays
Broad ETFs	SPY QQQ IWM	Core holdings, 1x
Sector ETFs	XLK XLF XLE XLV XLI XLY XLP XLU	Rotation universe, 1x
Thematic ETFs	SMH GLD TLT	SMH=semis, GLD=gold, TLT=long bonds

The One Rule That Matters More Than Anything Else

■ A RISK-OFF SWITCH beats a clever entry signal EVERY TIME for Calmar score. Going to cash when SPY is below SMA-200 eliminates the biggest drawdowns. This one if-statement does more for your score than any fancy indicator.

Official Strategy 1: AI/Chip Momentum

Hold strongest AI/chip names by 3-month return, but only above 50-day SMA.

```
# Universe: NVDA, AMD, MU, MRVL, AVGO, SMH

# Rule: Hold top 3-4 by 3-month return, only if above 50-day SMA, max 20% each

# Risk-off: Move to cash if position falls below its 50-day SMA

CHIPS = ['NVDA', 'AMD', 'MU', 'MRVL', 'AVGO', 'SMH']

def decide(market_state, portfolio_state, cash):
    scores = {}
    for t in CHIPS:
        bars = market_state.get(t, [])
        if len(bars) < 63: continue
        closes = [b['close'] for b in bars]
        sma50 = sum(closes[-50:]) / 50
        ret3m = (closes[-1] - closes[-63]) / closes[-63]
        if closes[-1] > sma50: # Only above SMA-50
            scores[t] = ret3m
    top = sorted(scores, key=scores.get, reverse=True)[:4]
    # ... equal weight into top, sell rest
```

Official Strategy 2: QQQ SMA Filter (Simplest Winning Bot)

Hold QQQ when above 100-day SMA. Go to cash otherwise. Dead simple. Very effective.

```
def decide(market_state, portfolio_state, cash):
    bars = market_state.get('QQQ', [])
    if len(bars) < 100: return []
    closes = [b['close'] for b in bars]
    sma100 = sum(closes[-100:]) / 100
    positions = portfolio_state.get('positions', [])
    held_qqq = next((p for p in positions if p['ticker']=='QQQ'), None)

    if closes[-1] > sma100: # Uptrend — hold QQQ
        if not held_qqq:
            price = portfolio_state['last_prices'].get('QQQ', 1)
            equity = cash + sum(p['quantity'] * portfolio_state['last_prices'].get(p['ticker'],0)
                                for p in positions)
            qty = int(equity * 0.95 / price) # 95% allocation
```

```

        if qty > 0: return [{'ticker':'QQQ','side':'buy','quantity':qty}]
    else: # Downtrend – go to cash
        if held_qqq:
            return [{'ticker':'QQQ','side':'sell','quantity':held_qqq['quantity']}]
    return []

```

Official Strategy 3: Sector Rotation (Most Popular Pick)

Every 5 days, hold top 3 sector ETFs by 3-month return. Go to cash if all negative.

```

SECTORS = ['XLK','XLF','XLE','XLV','XLI','XLY','XLP','XLU','SMH']

def decide(market_state, portfolio_state, cash):
    # Only rebalance every 5 trading days
    # (track day count externally or use a simple counter)

    # SPY risk-off switch
    spy = market_state.get('SPY', [])
    if len(spy) >= 200:
        spy_closes = [b['close'] for b in spy]
        if spy_closes[-1] < sum(spy_closes[-200:])/200:
            # Sell everything
            return [{'ticker':p['ticker'],'side':'sell','quantity':p['quantity']}
                    for p in portfolio_state.get('positions',[])]

    # Rank sectors by 3-month momentum
    mom = {}
    for s in SECTORS:
        bars = market_state.get(s,[])
        if len(bars) >= 63:
            c = [b['close'] for b in bars]
            ret = (c[-1]-c[-63])/c[-63]
            if ret > 0: mom[s] = ret # Only positive momentum

    if not mom: return [] # All negative – stay cash
    top3 = sorted(mom, key=mom.get, reverse=True)[:3]
    # Equal weight 30% each into top3...

```

RECOMMENDED STRATEGY: Combined Dual Momentum + Vol Target

This is the strategy most likely to produce a competitive Calmar score. It combines three proven components: absolute momentum filter, relative momentum ranking, and volatility-based position sizing. Plus the SPY SMA-200 risk-off switch.

Component 1: Risk-Off	If SPY < SMA-200 → sell everything → 100% cash. Non-negotiable.
Component 2: Absolute Mom	Only buy assets whose 12-month return > 0 (beats cash/T-bills).
Component 3: Relative Mom	Rank universe by 12-month return. Hold top 2-3.

Component 4: Vol Sizing	Size each position inversely to its 20-day volatility. Calmer = bigger position.
Component 5: Rebalance	Check and rebalance monthly (every ~21 trading days). Low turnover.
Universe	SPY, QQQ, IWM, GLD, TLT, XLK, XLF, XLE (8 assets — manageable)

05 — FULL agent.py — PRODUCTION READY

Copy this. Run preview.py. Submit.

■ This is the complete file to submit to Builderr.ai. Paste into agent.py in the template repo.

[illegible]


```

ranked = sorted(momentum, key=momentum.get, reverse=True)
top_n = ranked[:TOP_N]

# ■■ STEP 5: VOLATILITY-ADJUSTED POSITION SIZING ■■■■■■■■■■
# Inverse volatility weighting: calmer assets get bigger allocation
inv_vols = {t: 1.0 / volatility[t] for t in top_n}
total_inv = sum(inv_vols.values())
raw_weights = {t: inv_vols[t] / total_inv for t in top_n}

# Scale weights so portfolio hits target volatility
port_vol_est = sum(raw_weights[t] * volatility[t] for t in top_n)
vol_scalar = min(TARGET_VOL / port_vol_est, 1.5) if port_vol_est > 0 else 1.0

# Final weights capped at MAX_POS
final_weights = {t: min(raw_weights[t] * vol_scalar, MAX_POS) for t in top_n}

# ■■ STEP 6: SELL POSITIONS NOT IN TOP N ■■■■■■■■■■■■■■■■■■■■
for pos in positions:
    if pos['ticker'] not in top_n:
        orders.append({'ticker': pos['ticker'],
                       'side': 'sell', 'quantity': pos['quantity']})

# ■■ STEP 7: BUY TOP N WITH CORRECT SIZING ■■■■■■■■■■■■■■■■■■■■
total_equity = equity()
for ticker in top_n:
    target_val = total_equity * final_weights[ticker]
    price = prices.get(ticker, 0)
    if price <= 0:
        continue
    target_qty = int(target_val / price)
    current_qty = pos_map.get(ticker, {}).get('quantity', 0)
    diff_qty = target_qty - current_qty

    if diff_qty > 0 and cash >= diff_qty * price:
        orders.append({'ticker': ticker, 'side': 'buy', 'quantity': diff_qty})
        cash -= diff_qty * price
    elif diff_qty < 0:
        orders.append({'ticker': ticker, 'side': 'sell', 'quantity': abs(diff_qty)})

return orders

```

Run: `python preview.py` — should show PASS on all sample windows. If it fails, check leverage calculation and SPY data length.

backtest.py — Full Simulation Engine

```

"""backtest.py — Simulates agent.py logic on historical data"""
import numpy as np
from agent import decide

def run_backtest(price_data: dict, initial_cash: float = 100_000) -> dict:
    """
    price_data: {ticker: [{ts, open, high, low, close, volume}, ...]}
    Returns: {equity_curve, trades, daily_returns, final_metrics}
    """
    tickers = list(price_data.keys())
    if not tickers: return {}

    # Align all tickers to same date range
    all_dates = sorted(set(b['ts'] for t in tickers for b in price_data[t]))
    n_days = len(all_dates)

    cash = initial_cash
    positions = {} # {ticker: quantity}
    equity_curve = []
    trades = []

    for i, date in enumerate(all_dates):
        # Build market_state: last 220 bars up to today
        market_state = {}
        for t in tickers:
            bars_up_to_today = [b for b in price_data[t] if b['ts'] <= date]
            market_state[t] = bars_up_to_today[-220:]

        # Build last_prices
        last_prices = {}
        for t in tickers:
            bars = market_state.get(t, [])
            if bars: last_prices[t] = bars[-1]['close']

        # Build portfolio_state
        pos_list = [{'ticker': t, 'quantity': q,
                     'avg_cost': last_prices.get(t, 0)}
                    for t, q in positions.items() if q > 0]
        portfolio_state = {
            'cash': cash,
            'positions': pos_list,
            'last_prices': last_prices,
        }

        # Call decide()
        try:

```

```

        orders = decide(market_state, portfolio_state, cash)
except Exception as e:
    orders = [] # Bot crashed – do nothing

# Execute orders (market fill at today's close)
for order in (orders or []):
    t, side, qty = order['ticker'], order['side'], order['quantity']
    price = last_prices.get(t, 0)
    if price <= 0 or qty <= 0: continue

    if side == 'buy':
        cost = qty * price * 1.0005 # 0.05% slippage
        if cost <= cash:
            cash -= cost
            positions[t] = positions.get(t, 0) + qty
            trades.append({'date':date,'ticker':t,'side':'buy',
                           'qty':qty,'price':price})

    elif side == 'sell':
        held = positions.get(t, 0)
        sell_qty = min(qty, held)
        if sell_qty > 0:
            cash += sell_qty * price * 0.9995 # 0.05% slippage
            positions[t] = held - sell_qty
            trades.append({'date':date,'ticker':t,'side':'sell',
                           'qty':sell_qty,'price':price})

# Calculate portfolio value
pos_val = sum(positions.get(t,0) * last_prices.get(t,0) for t in tickers)
total_equity = cash + pos_val
equity_curve.append({'date': date, 'equity': round(total_equity, 2)})

return {
    'equity_curve': equity_curve,
    'trades': trades,
    'initial_cash': initial_cash,
    'final_equity': equity_curve[-1]['equity'] if equity_curve else initial_cash,
}

```

metrics.py — Complete Metrics Calculator

```

"""metrics.py – Calmar, Sharpe, Sortino, all metrics"""
import numpy as np

def calc_metrics(equity_curve: list) -> dict:
    """equity_curve: list of {'date': str, 'equity': float}"""
    if len(equity_curve) < 2: return {}

    values = [e['equity'] for e in equity_curve]
    eq = np.array(values, dtype=float)
    returns = np.diff(eq) / eq[:-1] # Daily returns

```


[illegible]

data.py — Price Fetcher (yfinance + fallback)

```
"""data.py — Fetch OHLCV from yfinance, format for backtest"""
import yfinance as yf
import json, os

CACHE_DIR = '.price_cache'
os.makedirs(CACHE_DIR, exist_ok=True)

def fetch_prices(tickers: list, start: str = None,
                 end: str = None, period: str = '1y') -> dict:
    result = {}
    for ticker in tickers:
        cache_key = f'{ticker}_{start}_{end}_{period}.json'
        cache_path = os.path.join(CACHE_DIR, cache_key)

        # Use cache if exists (avoid repeated API calls)
        if os.path.exists(cache_path):
            with open(cache_path) as f:
                result[ticker] = json.load(f)
            continue

        try:
            kw = dict(auto_adjust=True, progress=False)
            if start and end:
                df = yf.download(ticker, start=start, end=end, **kw)
            else:
                df = yf.download(ticker, period=period, **kw)

            if df.empty: continue

            # Flatten multi-index if present
            if hasattr(df.columns, 'levels'):
                df.columns = df.columns.get_level_values(0)

            bars = []
            for ts, row in df.iterrows():
                bars.append({
                    'ts':      str(ts.date()),
                    'open':    round(float(row.get('Open', row.get('open', 0))), 2),
                    'high':    round(float(row.get('High', row.get('high', 0))), 2),
                    'low':     round(float(row.get('Low', row.get('low', 0))), 2),
                    'close':   round(float(row.get('Close', row.get('close', 0))), 2),
                    'volume':  int( row.get('Volume', row.get('volume', 0))),
                })

            result[ticker] = bars
            with open(cache_path, 'w') as f:
                json.dump(bars, f)

        except Exception as e:
            print(f'Error fetching {ticker}: {e}')
```


main.py — Full FastAPI Server

[illegible]

```

        data = fetch_prices([ticker], period=period)
        return data.get(ticker, [])

@app.post('/signal/live')
def live_signal(req: LiveSignalReq):
    """What would the bot do today with live prices?"""
    prices = fetch_prices(req.tickers, period='1y')
    from agent import decide
    market_state = {t: prices[t][-220:] for t in req.tickers if prices.get(t)}
    last_prices = {t: market_state[t][-1]['close'] for t in market_state}
    portfolio = {'cash':100_000, 'positions':[], 'last_prices':last_prices}
    orders = decide(market_state, portfolio, 100_000)
    return {'orders': orders, 'last_prices': last_prices}

@app.get('/history')
def history(limit: int = 20):
    """Return saved backtest results from DB"""
    conn = sqlite3.connect('tradevision.db')
    rows = conn.execute(
        'SELECT id,created_at,tickers,calmar,sharpe,ann_return,max_dd'
        ' FROM backtests ORDER BY created_at DESC LIMIT ?', (limit,)
    ).fetchall()
    conn.close()
    return [{
        'id':r[0], 'date':r[1], 'tickers':r[2], 'calmar':r[3],
        'sharpe':r[4], 'ann_return':r[5], 'max_dd':r[6]} for r in rows]

@app.get('/strategies')
def strategies():
    return [
        {
            'id':'dual_momentum', 'name':'Dual Momentum', 'description':'12m momentum + SPY SMA-200 risk-off'
        },
        {
            'id':'sma_crossover', 'name':'SMA Crossover', 'description':'Golden/Death cross on QQQ'
        },
        {
            'id':'sector_rotation', 'name':'Sector Rotation', 'description':'Top 3 sectors monthly'
        },
        {
            'id':'vol_target', 'name':'Vol Targeting', 'description':'Inverse vol weighting'
        },
    ]

def _save_backtest(req, result):
    conn = sqlite3.connect('tradevision.db')
    conn.execute(
        '''CREATE TABLE IF NOT EXISTS backtests(
            id INTEGER PRIMARY KEY, created_at TEXT,
            tickers TEXT, start TEXT, end TEXT,
            calmar REAL, sharpe REAL, ann_return REAL, max_dd REAL,
            full_result TEXT)'''
    )
    m = result.get('metrics', {})
    conn.execute(
        'INSERT INTO backtests VALUES(NULL,datetime("now"),?,?,?,?,?,?,?,?,?)',
        (
            json.dumps(req.tickers), req.start_date, req.end_date,
            m.get('calmar_ratio',0), m.get('sharpe_ratio',0),
            m.get('ann_return_pct',0), m.get('max_drawdown_pct',0),

```


App.jsx — Root + Routing

```
import { useState } from 'react'
import Dashboard from './pages/Dashboard'
import Backtest from './pages/Backtest'
import Strategy from './pages/Strategy'
import Signals from './pages/Signals'

const NAV = [
  {id:'dashboard', label:'Dashboard', icon:'■'},
  {id:'backtest', label:'Backtest', icon:'■'},
  {id:'signals', label:'Signals', icon:'■'},
  {id:'strategy', label:'Strategy', icon:'■■'},
]

export default function App() {
  const [page, setPage] = useState('dashboard')
  return (
    <div style={{background:'#060B14',minHeight:'100vh',color:'#CDD9F0',
      fontFamily:'Inter,sans-serif'}}>

      {/* Sidebar */}
      <div style={{position:'fixed',top:0,left:0,width:200,height:'100vh',
        background:'#0D1525',borderRight:'1px solid #1E3A5F',padding:20}}>

        <h2 style={{color:'#F5A623',fontSize:16,marginBottom:32}}>
          TradeVision AI</h2>

        {NAV.map(n => (
          <div key={n.id} onClick={() => setPage(n.id)}
            style={{padding:'10px 12px',borderRadius:8,marginBottom:4,cursor:'pointer',
              background: page===n.id ? '#1A2940' : 'transparent',
              color: page===n.id ? '#00D4FF' : '#6B82A8',
              fontWeight: page===n.id ? 700 : 400}}>
            {n.icon} {n.label}
          </div>
        ))}

      </div>

      {/* Main */}
      <div style={{marginLeft:200,padding:24}}>
        {page==='dashboard' && <Dashboard />}
        {page==='backtest' && <Backtest />}
        {page==='signals' && <Signals />}
        {page==='strategy' && <Strategy />}
      </div>
    </div>
  )
}
```

```
}
```

Dashboard.jsx — Main Screen

```
import { useState, useEffect } from 'react'
import { LineChart, Line, XAxis, YAxis, Tooltip,
      ResponsiveContainer, ReferenceLine } from 'recharts'
import { runBacktest } from '../api/client'

export default function Dashboard() {
  const [data, setData] = useState(null)
  const [loading, setLoading] = useState(false)

  useEffect(() => { loadDemo() }, [])

  async function loadDemo() {
    setLoading(true)
    const res = await runBacktest({
      tickers: ['SPY', 'QQQ', 'IWM', 'XLK', 'GLD'],
      start_date: '2022-01-01', end_date: '2024-06-01',
      initial_cash: 100000
    })
    setData(res)
    setLoading(false)
  }

  if (loading) return <div style={{color:'#00D4FF',padding:40}}>Loading...</div>
  if (!data) return null
  const m = data.metrics || {}

  const METRICS = [
    {label:'Calmar Ratio', value: m.calmar_ratio, color:'#00FF88',
      sub:'Target > 2.0'},
    {label:'Ann. Return', value: m.ann_return_pct+'%', color:'#00D4FF'},
    {label:'Max Drawdown', value: '-' + m.max_drawdown_pct+'%', color:'#FF4455'},
    {label:'Sharpe Ratio', value: m.sharpe_ratio, color:'#9B6DFF'},
    {label:'Win Rate', value: m.win_rate_pct+'%', color:'#F5A623'},
    {label:'Profit Factor',value: m.profit_factor, color:'#00FF88'},
  ]

  return (
    <div>
      <h1 style={{color:'white',fontSize:22,marginBottom:20}}>
        Portfolio Dashboard</h1>
      { /* Metrics Grid */ }
      <div style={{display:'grid',gridTemplateColumns:'repeat(3,1fr)',gap:12,
        marginBottom:24}}>
        {METRICS.map(m => (
          <div key={m.label} style={{background:'#111E30',borderRadius:12,
            padding:'16px 20px',border:'1px solid #1E3A5F'}}>
```



```

        <p style={{color:'#6B82A8',fontSize:11,textTransform:'uppercase',
            letterSpacing:1,margin:0}}>{m.label}</p>
        <p style={{color:m.color,fontSize:28,fontWeight:700,margin:'4px 0 0'}}>
            {m.value}</p>
        {m.sub && <p style={{color:'#6B82A8',fontSize:11,margin:0}}>{m.sub}</p>}
    </div>
    )))
</div>
/* Equity Curve */
<div style={{background:'#111E30',borderRadius:12,padding:20,
    border:'1px solid #1E3A5F',marginBottom:20}}>
    <h3 style={{color:'#00D4FF',margin:'0 0 16px'}}>Equity Curve</h3>
    <ResponsiveContainer width='100%' height={260}>
        <LineChart data={data.equity_curve}>
            <XAxis dataKey='date' tick={{fill:'#6B82A8',fontSize:10}}
                tickFormatter={d=>d.slice(0,7)} />
            <YAxis tick={{fill:'#6B82A8',fontSize:10}}
                tickFormatter={v=>('$'+(v/1000).toFixed(0)+'k')} />
            <Tooltip formatter={v=>['$'+v.toLocaleString(),'Equity']}
                contentStyle={{background:'#0D1525',border:'1px solid #1E3A5F'}} />
            <ReferenceLine y={100000} stroke='#1E3A5F' strokeDasharray='4 4' />
            <Line type='monotone' dataKey='equity' stroke='#6366F1'
                strokeWidth={2} dot={false} />
        </LineChart>
    </ResponsiveContainer>
</div>
</div>
)
}

```

api/client.js — All API Calls

```

const BASE = import.meta.env.VITE_API_URL || 'http://localhost:8000'

const post = (url, body) =>
    fetch(BASE+url, {method:'POST',
        headers:{'Content-Type':'application/json'},
        body:JSON.stringify(body)}).then(r=>r.json())

const get = (url) => fetch(BASE+url).then(r=>r.json())

export const runBacktest = (params) => post('/backtest', params)
export const getLiveSignal = (tickers) => post('/signal/live', {tickers})
export const getPrices = (ticker, period='1y') =>
    get(`/prices/${ticker}?period=${period}`)
export const getHistory = (limit=20) => get(`/history?limit=${limit}`)
export const getStrategies = () => get('/strategies')

```


09 — DATABASE DESIGN (SQLite)

Using SQLite — zero setup, file-based, perfect for solo dev. Stores backtest results, trade history, and strategy configs.

Schema — All Tables

```
-- db_schema.sql

-- Run: python -c "import sqlite3; conn=sqlite3.connect('tradevision.db'); conn.executescript(open('db_schema.sql').read())"

CREATE TABLE IF NOT EXISTS backtests (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    created_at    TEXT      NOT NULL DEFAULT (datetime('now')),
    strategy      TEXT      NOT NULL DEFAULT 'dual_momentum',
    tickers       TEXT      NOT NULL, -- JSON array
    start_date    TEXT      NOT NULL,
    end_date      TEXT      NOT NULL,
    initial_cash  REAL       NOT NULL DEFAULT 100000,
    -- Key metrics (denormalized for fast queries)
    calmar_ratio  REAL,
    sharpe_ratio  REAL,
    sortino_ratio REAL,
    ann_return_pct REAL,
    max_drawdown_pct REAL,
    win_rate_pct  REAL,
    profit_factor REAL,
    n_trades      INTEGER,
    -- Full result blob
    equity_curve  TEXT, -- JSON array
    full_result   TEXT  -- JSON blob
);

CREATE TABLE IF NOT EXISTS trades (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    backtest_id   INTEGER REFERENCES backtests(id),
    trade_date    TEXT      NOT NULL,
    ticker        TEXT      NOT NULL,
    side          TEXT      NOT NULL CHECK(side IN ('buy','sell')),
    quantity      INTEGER NOT NULL,
    price         REAL       NOT NULL,
    value         REAL       GENERATED ALWAYS AS (quantity * price) VIRTUAL
);

CREATE TABLE IF NOT EXISTS strategy_configs (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    name          TEXT      NOT NULL UNIQUE,
    config_json   TEXT      NOT NULL, -- JSON of parameters
    created_at    TEXT      DEFAULT (datetime('now')),
```

```
        is_active    INTEGER DEFAULT 0
    );

CREATE TABLE IF NOT EXISTS price_cache (
    ticker           TEXT NOT NULL,
    trade_date       TEXT NOT NULL,
    open             REAL, high REAL, low REAL, close REAL, volume INTEGER,
    fetched_at       TEXT DEFAULT (datetime('now')),
    PRIMARY KEY (ticker, trade_date)
);

-- Indexes for performance
CREATE INDEX IF NOT EXISTS idx_backtests_calmar ON backtests(calmar_ratio DESC);
CREATE INDEX IF NOT EXISTS idx_trades_backtest  ON trades(backtest_id);
CREATE INDEX IF NOT EXISTS idx_prices_ticker    ON price_cache(ticker, trade_date);
```

10 — COMPLETE API CONTRACTS

Method	Endpoint	Request Body	Response	Status codes
GET	/health	—	{status,version}	200
POST	/backtest	tickers,start,end,cash,strategy	{equity_curve,trades,metrics,elapsed}	200,400,500
GET	/prices/{ticker}	period (query)	[[ts,open,high,low,close,volume]]	200,404
POST	/signal/live	tickers	{orders,last_prices}	200,500
GET	/history	limit (query, default 20)	[[{id,date,tickers,calmar,...}]]	200
GET	/strategies	—	[[{id,name,description}]]	200

Full Backtest Response Shape

```
{
  "equity_curve": [
    {"date": "2022-01-03", "equity": 100000.0},
    {"date": "2022-01-04", "equity": 99821.5},
    ...
  ],
  "trades": [
    {"date": "2022-02-01", "ticker": "QQQ", "side": "buy", "qty": 42, "price": 358.20},
    {"date": "2022-03-15", "ticker": "QQQ", "side": "sell", "qty": 42, "price": 341.10},
    ...
  ],
  "metrics": {
    "total_return_pct": 18.4,
    "ann_return_pct": 14.2,
    "max_drawdown_pct": 6.8,
    "calmar_ratio": 2.09,
    "sharpe_ratio": 1.74,
    "sortino_ratio": 2.31,
    "win_rate_pct": 57.3,
    "profit_factor": 1.82,
    "best_day_pct": 3.1,
    "worst_day_pct": -2.4,
    "n_days": 371,
    "drawdown_series": [0, -0.3, -1.1, ...]
  },
  "initial_cash": 100000,
  "final_equity": 118400,
  "elapsed_sec": 2.3
}
```

11 — DEPLOYMENT — RAILWAY + VERCEL (FREE)

requirements.txt

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
yfinance==0.2.40
pandas==2.2.2
numpy==1.26.4
python-dotenv==1.0.1
pydantic==2.7.0
```

Backend → Railway

1. Push	cd backend && git init && git add . && git commit -m 'init' && git push origin main
2. Deploy	railway.app → New Project → Deploy from GitHub → select backend repo
3. Port	Add env var: PORT=8000 in Railway dashboard
4. URL	Your API: https://tradevision-api.up.railway.app (copy this)
5. Procfile	Create file: Procfile with content: web: uvicorn main:app --host 0.0.0.0 --port \$PORT

Frontend → Vercel

1. .env	Create frontend/.env: VITE_API_URL= https://tradevision-api.up.railway.app
2. Push	cd frontend && git init && git push origin main
3. Deploy	vercel.com → Import project → select frontend repo
4. Build cmd	npm run build
5. Output dir	dist
6. Env var	Add VITE_API_URL in Vercel dashboard → Environment Variables
7. URL	Your app: https://tradevision.vercel.app

Local Development

```
# Terminal 1 – Backend
cd backend
pip install -r requirements.txt
python main.py
# → Running at http://localhost:8000
# → API docs at http://localhost:8000/docs

# Terminal 2 – Frontend
cd frontend
```


12 — COMPLETE SUBMISSION CHECKLIST

■ Do these in order. Do not skip. Run `preview.py` before every submission.

TODAY	Clone repo	<code>git clone https://github.com/builderr-ai/builderr-trading-template && cd builderr-trading-template</code>
TODAY	First test	<code>python preview.py</code> # Should PASS with baseline.py in ~10 seconds
TODAY	Add your strategy	Copy <code>agent.py</code> code from Section 05 of this doc into <code>agent.py</code>
TODAY	Test your strategy	<code>python preview.py</code> # Must PASS before you submit
TODAY	Run selfcheck	<code>python selfcheck.py</code> # Checks for secrets + syntax errors
TODAY	Push to GitHub	<code>git add . && git commit -m 'TradeVision dual momentum' && git push</code>
TODAY	EMAIL SUBMISSION	Send repo URL to <code>submit@builderr.ai</code> – Subject: 'Round 1 submission'
DAY 2	Check email	Builderr emails robustness profile same day. Read it carefully.
DAY 2	Improve if needed	If robustness profile shows weakness, tweak strategy (have 3 more revisions)
ONGOING	Monitor leaderboard	Daily leaderboard on <code>builderr.ai</code> – track your Calmar vs others
WEEK 2	Optional revision	If you have a better idea, revise and resubmit (up to 4 total)
JULY 2	Contest ends	Round 1 closes. Re-run begins. Winners announced after.

13 — EVERY TRAP & HOW TO AVOID IT

LOOKAHEAD BIAS	Using today's close price to make today's decision. Example: <code>bar[-1]</code> is today's close but <code>decide()</code> is called before market close. PREVENTION: always use <code>bar[-2]</code> or earlier for same-day signals. Only use bars up to yesterday's close.	INSTANT DQ
CURVE FITTING	Optimizing parameters until historical backtest looks perfect. Dies on live data and re-run. PREVENTION: test $\pm 20\%$ on all parameters. If performance collapses, you overfit. Use simple strategies with 1-3 parameters.	Fails re-run
OVER-LEVERAGE	Betting max size for max return. 1.5x cap auto-flattens you. PREVENTION: Calculate total beta-adjusted exposure before every trade. Leave 5% buffer under 1.5x limit.	Auto-flatten + DQ
CONCENTRATION	Holding >30% in one position for >5 days. PREVENTION: Hard-cap each position at 29% in code. Check on every rebalance.	DQ risk
API KEYS IN REPO	If using LLM signals, never commit API keys to public repo. PREVENTION: Use <code>.env</code> file + <code>.gitignore</code> . Or use private repo. Or endpoint mode.	Key leaked to internet
decide() TIMEOUT	Function taking >5 seconds per call. PREVENTION: Time your function locally: <code>import time; t=time.time(); decide(...); print(time.time()-t)</code> . Must be <2 seconds to be safe.	No orders submitted
LEVERAGED ETF DECAY	TQQQ/SOXL reset daily. $-10\%/+10\%$ = net loss due to volatility decay. They lost 82% in 2022. PREVENTION: Use leveraged ETFs tactically only during calm uptrends. Never buy-and-hold.	Portfolio destruction
NO RISK-OFF	Not going to cash during downtrends. Max drawdown kills Calmar. PREVENTION: ALWAYS include SPY SMA-200 check as first line of <code>decide()</code> . This is the #1 Calmar improver.	Poor Calmar score
OFF-UNIVERSE TICKERS	Ordering a ticker not in <code>universe.json</code> . Orders silently ignored. PREVENTION: Filter all orders against the official universe list. Download <code>universe.json</code> from repo.	Capital sits unused
RSI-2 TRAP	RSI-2 has 75%+ win rate but occasional brutal drawdowns destroy Calmar. PREVENTION: Study it but don't submit it raw. The official guide explicitly warns about this.	Poor Calmar

14 — FINANCE KNOWLEDGE THAT MATTERS FOR WINNING

Only what you need. Nothing more.

The Only Metric That Wins: Calmar Ratio

$\text{Calmar} = \text{Annualized Return} / \text{Max Drawdown}$

To win: maximize return, MINIMIZE DRAWDOWN even more

A 8% return with 2% drawdown (Calmar 4.0) BEATS 25% return with 20% drawdown (Calmar 1.25)

The 5 Concepts You Must Understand

1. SMA-200	Simple Moving Average of last 200 days. If SPY price > SMA-200 = uptrend. If below = downtrend. This single check prevents the worst drawdowns.
2. Momentum	Rank assets by past 3-12 month return. Buy winners, ignore losers. Works because trends persist. Simple and proven.
3. Volatility	Standard deviation of daily returns $\times \sqrt{252}$. Higher vol = more risk. Size positions SMALLER when vol is HIGH.
4. Max Drawdown	Biggest peak-to-trough loss. This is the denominator of Calmar. Going to cash avoids drawdowns. Going to cash is ALWAYS right when SPY < SMA-200.
5. Position Sizing	Never go all-in on one bet. 25-30% max per position. Use volatility to size — calmer assets get more allocation.

Why Going to Cash is Your Best Friend

In a competition scored by Calmar ratio, cash IS a position. Cash has zero volatility and zero drawdown. Any day you're in cash and the market drops is a day your Calmar score improves relative to fully-invested bots.

Scenario	Fully Invested Bot	Cash Bot	Winner
Market drops 10%	Portfolio -10%	Cash: 0%	Cash bot — better Calmar
Market rises 5%	Portfolio +5%	Cash: 0%	Invested bot — higher return but...
Net over 1 month	Maybe flat	Definitely flat	Depends on magnitude
Key insight	Both may tie on return	Cash has ZERO drawdown	Cash wins on Calmar

■ The strategy that wins Builderr.ai is the one that makes money in uptrends and DISAPPEARS in downtrends.

15 — 7-DAY BUILD PLAN (HOUR BY HOUR)

■ **PRIORITY ORDER:** Submit `agent.py` TODAY first. Build the dashboard second. The contest doesn't care about the dashboard. Your portfolio does.

DAY 1 — TODAY

0-30 min	Clone <code>buildderr</code> template. Run <code>python preview.py</code> with <code>baseline.py</code> . See PASS.
30-90 min	Copy <code>agent.py</code> from Section 05. Run <code>preview.py</code> again. Must PASS.
90-120 min	Run <code>selfcheck.py</code> . Fix any issues. Push to GitHub.
120 min	EMAIL <code>submit@buildderr.ai</code> with your repo URL. YOU ARE SUBMITTED.
Rest of day	Set up <code>backend/</code> folder. Install requirements. Test <code>main.py</code> runs.

DAY 2

Morning	Read robustness profile email from Builderr. Note weak spots.
Morning	Complete <code>backtest.py</code> and <code>metrics.py</code> from Section 06.
Afternoon	Complete <code>data.py</code> with <code>yfinance</code> . Test fetching SPY, QQQ prices.
Evening	Test full backtest pipeline: <code>fetch</code> → <code>backtest</code> → <code>metrics</code> . Print results.

DAY 3

Morning	Set up React project: <code>npm create vite@latest frontend -- --template react</code>
Morning	Install: <code>npm install recharts axios tailwindcss</code>
Afternoon	Build <code>App.jsx</code> with sidebar navigation (4 pages).
Evening	Build <code>Dashboard.jsx</code> with <code>MetricCard</code> components and equity curve chart.

DAY 4

All day	Build <code>Backtest.jsx</code> — form inputs, POST <code>/backtest</code> , show results.
All day	Wire <code>api/client.js</code> . Test frontend ↔ backend connection.
Evening	Fix CORS issues. Test full flow locally.

DAY 5

Morning	Build <code>Signals.jsx</code> — live signal display, <code>last_prices</code> table.
Afternoon	Build <code>Strategy.jsx</code> — strategy selector, parameter display.
Evening	Add drawdown chart to Dashboard using the <code>drawdown_series</code> from metrics.

DAY 6

- | | |
|------------------|---|
| Morning | Deploy backend to Railway. Get URL. |
| Afternoon | Deploy frontend to Vercel with VITE_API_URL set. |
| Evening | Test full app on live deployment. Fix any issues. |

DAY 7

- | | |
|------------------|--|
| Morning | Polish UI — colors, spacing, loading states. |
| Afternoon | Optional: Consider strategy revision for Builderr (have 3 more submissions). |
| Evening | Done. Share the app URL. Add to portfolio/GitHub README. |

THE GOD DOC IS COMPLETE.

15 Sections · Every Rule · Every Line of Code · Every Formula · Every Trap

Built for Meet — June 2026 — Go win.